```python
"""Requirement-level tests for SentinelAI's five assessment criteria."""

from __future__ import annotations

from datetime import datetime, timedelta
import tempfile
import unittest
from pathlib import Path

import pandas as pd

from src import detector, features


def event(
    entity_id: str,
    timestamp: datetime,
    resource: str = "email",
    command: str = "login > read > logout",
    source_ip: str = "10.1.1.1",
) -> dict:
    return {
        "entity_id": entity_id,
        "entity_type": "user",
        "timestamp": timestamp,
        "source_ip": source_ip,
        "geo_location": "Bengaluru, IN",
        "geo_lat": 12.9716,
        "geo_lon": 77.5946,
        "resource_accessed": resource,
        "auth_result": "success",
        "device_fingerprint": "managed-device",
        "command_sequence": command,
        "bytes_transferred": 1000,
        "hour": timestamp.hour,
        "is_anomaly": 0,
    }


class RequirementTests(unittest.TestCase):
    def build_profiles(self, data: pd.DataFrame):
        original_path = features.BASELINE_PATH
        with tempfile.TemporaryDirectory() as directory:
            features.BASELINE_PATH = Path(directory) / "profiles.csv"
            result = features.build_baselines(data)
        features.BASELINE_PATH = original_path
        return result

    def test_sequential_features_capture_bursts_and_impossible_travel(self) -> None:
        start = datetime(2026, 1, 1, 9)
        data = pd.DataFrame(
            [
                event("u1", start),
                event("u1", start + timedelta(minutes=30)),
                {
                    **event("u1", start + timedelta(minutes=60)),
                    "geo_lat": 40.7128,
                    "geo_lon": -74.0060,
                },
            ]
        )
        result = features.add_temporal_features(data)
        self.assertEqual(result.iloc[-1]["source_ip_attempts_1h"], 3)
        self.assertEqual(result.iloc[-1]["impossible_travel_flag"], 1)

    def test_imbalance_model_uses_balanced_class_weights(self) -> None:
        model = detector.make_random_forest()
        self.assertEqual(
            model.named_steps["classifier"].class_weight,
            "balanced_subsample",
        )

    def test_confirmed_safe_drift_is_adapted(self) -> None:
        start = datetime(2026, 1, 1, 10)
        data = pd.DataFrame(
            [
                event("u1", start),
                event("u1", start + timedelta(days=1)),
                event("u1", start + timedelta(days=15), "new_a"),
                event("u1", start + timedelta(days=16), "new_b"),
                event("u1", start + timedelta(days=17), "new_c"),
                event("u1", start + timedelta(days=18), "new_a"),
            ]
        )
        baselines, population, _ = self.build_profiles(data)
        result = features.add_baseline_features(data, baselines, population)
        self.assertEqual(result.iloc[4]["concept_drift_adapted"], 1)
        self.assertEqual(result.iloc[5]["new_resource"], 0)

    def test_alert_explanation_contains_concrete_factor(self) -> None:
        row = pd.Series(
            {
                "impossible_travel_flag": 1,
                "distance_from_previous_km": 800,
                "failed_auth": 0,
                "source_ip_attempts_1h": 1,
                "source_ip_unique_entities_1h": 1,
                "new_device": 0,
                "new_location": 0,
                "new_source_ip": 0,
                "new_resource": 0,
                "new_command_sequence": 0,
                "is_off_hours": 0,
                "bytes_zscore": 0,
            }
        )
        self.assertIn("impossible travel", detector.explain_event(row))
        self.assertEqual(detector.risk_tier(80), "Critical")

    def test_cold_start_uses_population_baseline(self) -> None:
        start = datetime(2026, 1, 1, 10)
        data = pd.DataFrame(
            [
                event("known", start),
                event("known", start + timedelta(days=1)),
                event("new_user", start + timedelta(days=16), source_ip="10.9.9.9"),
            ]
        )
        baselines, population, _ = self.build_profiles(data)
        result = features.add_baseline_features(data, baselines, population)
        self.assertEqual(result.iloc[-1]["baseline_source"], "population")
        self.assertGreater(result.iloc[-1]["baseline_event_count"], 0)
```

```python
    def test_evaluation_split_is_chronological(self) -> None:
        timestamps = pd.date_range("2026-01-01", periods=8, freq="D")
        data = pd.DataFrame(
            {"timestamp": timestamps, "is_anomaly": [0, 0, 1, 0, 0, 1, 0, 1]}
        )
        train, test = detector.temporal_train_test_split(data)
        self.assertLess(train["timestamp"].max(), test["timestamp"].min())

    def test_attack_signature_overrides_generic_classifier(self) -> None:
        class GenericModel:
            def predict(self, values):
                return ["brute_force"] * len(values)

        row = {feature: 0 for feature in detector.NUMERIC_FEATURES}
        row.update({feature: "unknown" for feature in detector.CATEGORICAL_FEATURES})
        row.update(
            {
                "failed_auth": 1,
                "source_ip_unique_entities_1h": 12,
                "source_ip_attempts_1h": 12,
            }
        )
        result = detector.predict_attack_types(pd.DataFrame([row]), GenericModel())
        self.assertEqual(result[0], "credential_stuffing")

    def test_safe_cold_start_does_not_become_an_alert(self) -> None:
        data = pd.DataFrame(
            [
                {
                    "cold_start_entity": 1,
                    "failed_auth": 0,
                    "impossible_travel_flag": 0,
                    "new_device": 0,
                    "source_ip_attempts_1h": 1,
                    "bytes_zscore": 0.5,
                    "is_off_hours": 0,
                }
            ]
        )
        score = detector.apply_cold_start_guard(data, [82.0])
        self.assertEqual(score[0], 20.0)
        self.assertEqual(data.iloc[0]["cold_start_safe_context"], 1)


if __name__ == "__main__":
    unittest.main()
```